

Neural Computation Coursework

Group 30

Contents

Introduction	3
Implementation	4
Fully Convolutional Network (FCN)	4
Outline	4
Architecture	4
Implementation	5
Experimentation	6
U-Net	7
Outline	7
Architecture	7
Findings	9
Implementation of U-Net	9
Experiments	11
Conclusion	14
References	14

Introduction

For this assignment, our task was to develop a convolutional neural network for segmentation of cardiac MR images into four regions: background, right ventricle, myocardium, and left ventricle.

To produce this network, we will be utilising a dataset of 200 images, with 100 being used for training, 20 for validation and 80 for testing. For most of the data we have received, we have a pair of two images: the original scan image and an output mask done by a person manually. The training set will be used to train the model's parameters to fit it for a segmentation task for images of this type. The validation set will then be used to tweak the hyper-parameters that define the way the network trains its own parameters by comparing models that use different values for these hyper-parameters. The purpose of this is to use unseen data to compare how well the various models generalise to them.

For the test image dataset, we received only the original image and not a corresponding mask. These will be used to evaluate how the trained network generalises with its hyper-parameters tuned. This ensures that the hyper-parameters chosen do not happen to overfit for the validation dataset.

The coursework objective was to compare segmentation architectures and select the strongest model under the supplied validation and held-out test setup. This educational experiment did not assess clinical safety or readiness.

Implementation

We compared two established semantic-segmentation architectures for the supplied task: a fully convolutional network (FCN-8) and U-Net.

Fully Convolutional Network (FCN)

Outline

Fully convolutional networks can accept spatial inputs and produce dense spatial predictions. We implemented an FCN-8 variant as one baseline for the coursework comparison.

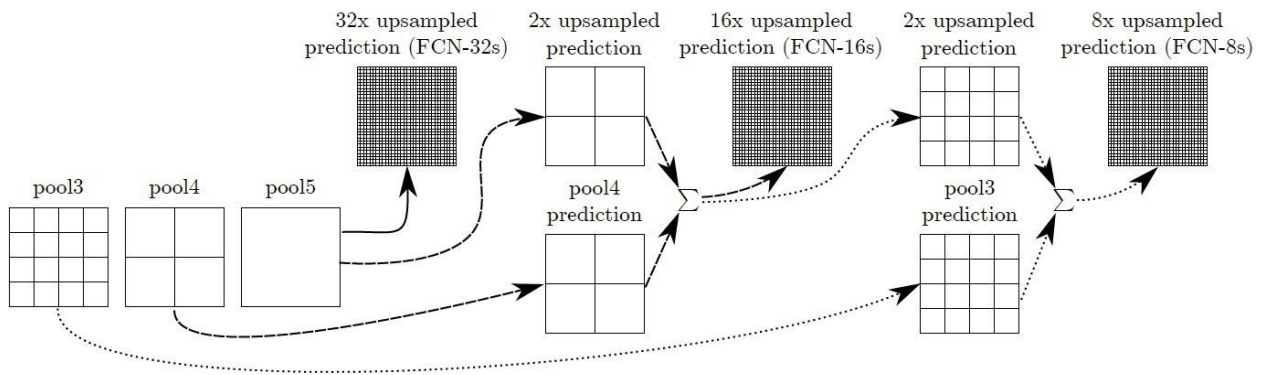


Figure 1: Fusing for FCN-16s and FCN-8s (Tsang, 2018)

Architecture

The diagram above illustrates the core design of FCNs. Starting off with FCN-32, we take the fifth pooling layer (*pool5*) and up-sample it to produce an output that is then up-sampled 32 times to serve as the predicted output for this particular FCN version (32). Next, we add up a 2x up-sampled prediction of the fifth pooling layer (*pool5*) along with the fourth pooling layer (*pool4*) (*sum1*) to produce a 16x up-sampled prediction for FCN-16. Finally, the third pooling layer (*pool3*) is summed up with the 2x up-sampled *sum1* to produce the 8x up-sampled predicted output for FCN-8.

Implementation

The PyTorch implementation used five convolutional stages followed by convolutional classifier layers, with skip fusion from intermediate feature maps. The experiments used cross-entropy loss and evaluated Adam with a learning rate of 0.001 among the reported configurations.

We did not apply a custom initialisation scheme; the layers retained PyTorch's default parameter initialisation and were then optimised during training.

We translated the aforementioned diagram (*Figure 1*) to Python (Huan, 2020) to arrive at the FCN-8 model as seen below:

```
from torch import nn

class FCN8(nn.Module):
    def __init__(self, num_classes=4):
        super().__init__()

        self.conv1_1 = nn.Conv2d(1, 64, kernel_size=3, padding=1)
        self.relu1_1 = nn.ReLU(inplace=True)
        self.conv1_2 = nn.Conv2d(64, 64, kernel_size=3, padding=1)
        self.relu1_2 = nn.ReLU(inplace=True)
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2, ceil_mode=True)

        self.conv2_1 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.relu2_1 = nn.ReLU(inplace=True)
        self.conv2_2 = nn.Conv2d(128, 128, kernel_size=3, padding=1)
        self.relu2_2 = nn.ReLU(inplace=True)
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2, ceil_mode=True)

        self.conv3_1 = nn.Conv2d(128, 256, kernel_size=3, padding=1)
        self.relu3_1 = nn.ReLU(inplace=True)
        self.conv3_2 = nn.Conv2d(256, 256, kernel_size=3, padding=1)
        self.relu3_2 = nn.ReLU(inplace=True)
        self.conv3_3 = nn.Conv2d(256, 256, kernel_size=3, padding=1)
        self.relu3_3 = nn.ReLU(inplace=True)
        self.pool3 = nn.MaxPool2d(kernel_size=2, stride=2, ceil_mode=True)

        self.conv4_1 = nn.Conv2d(256, 512, kernel_size=3, padding=1)
        self.relu4_1 = nn.ReLU(inplace=True)
        self.conv4_2 = nn.Conv2d(512, 512, kernel_size=3, padding=1)
        self.relu4_2 = nn.ReLU(inplace=True)
        self.conv4_3 = nn.Conv2d(512, 512, kernel_size=3, padding=1)
        self.relu4_3 = nn.ReLU(inplace=True)
        self.pool4 = nn.MaxPool2d(kernel_size=2, stride=2, ceil_mode=True)

        self.conv5_1 = nn.Conv2d(512, 512, kernel_size=3, padding=1)
        self.relu5_1 = nn.ReLU(inplace=True)
        self.conv5_2 = nn.Conv2d(512, 512, kernel_size=3, padding=1)
        self.relu5_2 = nn.ReLU(inplace=True)
        self.conv5_3 = nn.Conv2d(512, 512, kernel_size=3, padding=1)
        self.relu5_3 = nn.ReLU(inplace=True)
        self.pool5 = nn.MaxPool2d(kernel_size=2, stride=2, ceil_mode=True)

        self.fc6 = nn.Conv2d(512, 4096, kernel_size=7)
        self.relu6 = nn.ReLU(inplace=True)
        self.drop6 = nn.Dropout2d()
        self.fc7 = nn.Conv2d(4096, 4096, kernel_size=1)
        self.relu7 = nn.ReLU(inplace=True)
        self.drop7 = nn.Dropout2d()

        self.score_fr = nn.Conv2d(4096, num_classes, kernel_size=1)
        self.score_pool4 = nn.Conv2d(512, num_classes, kernel_size=1)
        self.score_pool3 = nn.Conv2d(256, num_classes, kernel_size=1)
        self.upscore2 = nn.ConvTranspose2d(
            num_classes, num_classes, kernel_size=4, stride=2, bias=False
        )
        self.upscore_pool4 = nn.ConvTranspose2d(
            num_classes, num_classes, kernel_size=4, stride=2, bias=False
        )
        )
```

```

        self.upscore8 = nn.ConvTranspose2d(
            num_classes, num_classes, kernel_size=16, stride=8, bias=False
        )

    def forward(self, x):
        h = x
        h = self.relu1_1(self.conv1_1(h))
        h = self.relu1_2(self.conv1_2(h))
        h = self.pool1(h)

        h = self.relu2_1(self.conv2_1(h))
        h = self.relu2_2(self.conv2_2(h))
        h = self.pool2(h)

        h = self.relu3_1(self.conv3_1(h))
        h = self.relu3_2(self.conv3_2(h))
        h = self.relu3_3(self.conv3_3(h))
        h = self.pool3(h)
        pool3 = h

        h = self.relu4_1(self.conv4_1(h))
        h = self.relu4_2(self.conv4_2(h))
        h = self.relu4_3(self.conv4_3(h))
        h = self.pool4(h)
        pool4 = h

        h = self.relu5_1(self.conv5_1(h))
        h = self.relu5_2(self.conv5_2(h))
        h = self.relu5_3(self.conv5_3(h))
        h = self.pool5(h)

        h = self.relu6(self.fc6(h))
        h = self.drop6(h)

        h = self.relu7(self.fc7(h))
        h = self.drop7(h)

        h = self.score_fr(h)
        h = self.upscore2(h)
        upscore2 = h # 32x

        h = self.score_pool4(pool4)
        h = h[:, :, 5 : 5 + upscore2.size()[2], 5 : 5 + upscore2.size()[3]]
        score_pool4c = h

        # Adding upsampled prediction with pool4 prediction for 16x upsampled prediction
        h = upscore2 + score_pool4c
        h = self.upscore_pool4(h)
        upscore_pool4 = h

        h = self.score_pool3(pool3)
        h = h[:, :, 9 : 9 + upscore_pool4.size()[2], 9 : 9 + upscore_pool4.size()[3]]
        score_pool3c = h

        # Adding pool4 (pool1 + pool4) and pool3
        h = upscore_pool4 + score_pool3c
        h = self.upscore8(h)
        h = h[:, :, 31 : 31 + x.size()[2], 31 : 31 + x.size()[3]].contiguous()

    return h

```

The following code is taken from the portion of the code where the model is initially designed. After the five convolution layers and two fully-connected layers are defined, the forward function displayed below is used to upscore different layers and compute the corresponding sums (as described in the *Architecture* section above):

Experimentation

We trained the FCN candidates on the supplied training set for up to 100 epochs and monitored performance on held-out data. Longer training improved the

training fit but did not consistently improve validation performance. We also evaluated dropout in the classifier layers as a regularisation choice.

U-Net

Outline

U-Net is an encoder-decoder convolutional network developed for biomedical image segmentation. Its skip connections pass encoder feature maps to corresponding decoder stages, combining spatial detail with progressively aggregated features.

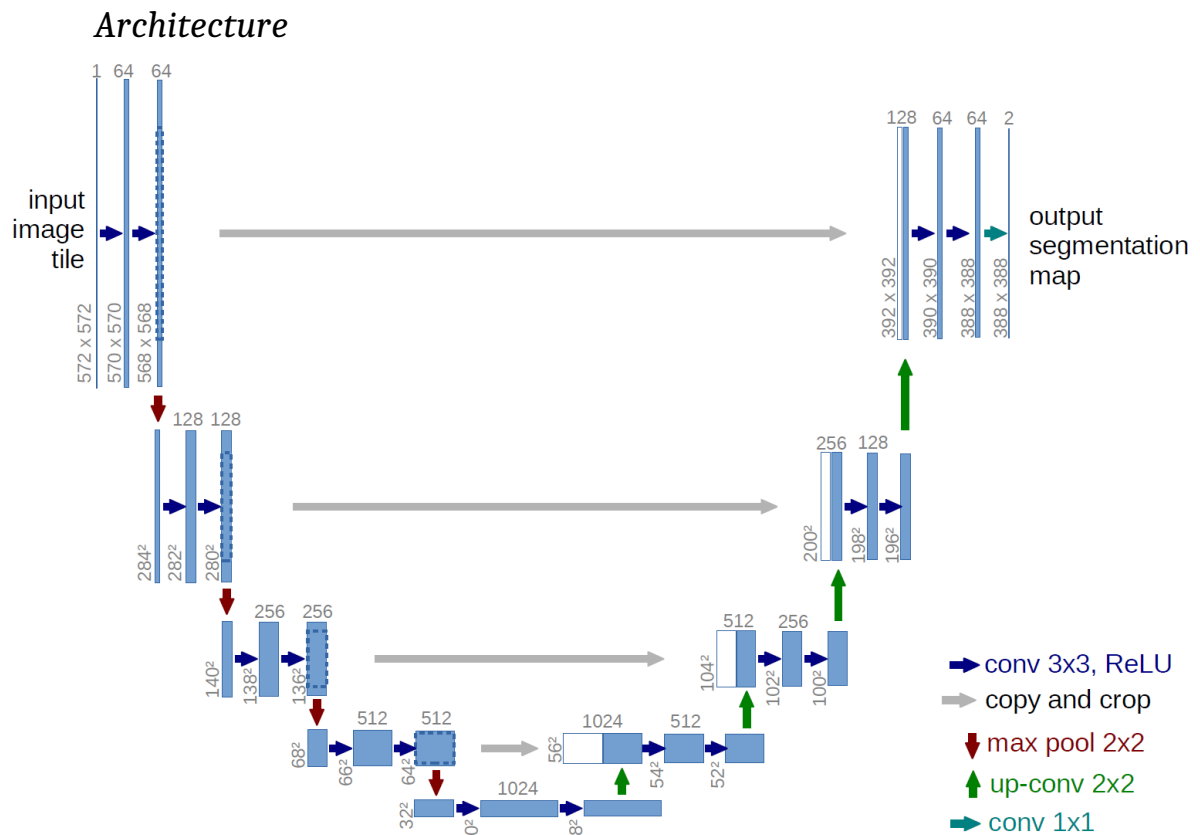


Figure 2: U-Net architecture - (Ronneberger et al., 2011)

The input image is initially fed into the network at the beginning. The data is propagated through the network along all possible paths and outputs a ready segmentation map. Each box represents a multi-channel feature map with its corresponding size (at the bottom) and the number of feature channels (at the top).

The main process that the model carries out is convolutions followed by a non-linear activation function (ReLU).

The original U-Net paper used valid 3 x 3 convolutions followed by ReLU, reducing the spatial extent by one pixel on each side per convolution. The submitted implementation instead used padding 1 to preserve spatial dimensions within each block.

A max pooling function is then applied to reduce the size/resolution of the feature map. This acts on each channel separately, iteratively processing the single maximum activation in a (2x2) frame.

After each max-pooling operation, the implementation doubled the number of feature channels while reducing spatial resolution. The contracting path therefore extracted increasingly broad image context for dense prediction.

At the deepest feature map, each decoder stage first applies a transposed convolution to increase spatial resolution. The upsampled features are then concatenated with the corresponding encoder feature map and passed through the double-convolution block; repeating these stages recovers the output resolution.

The key U-Net operation is this skip connection: each decoder stage concatenates the upsampled representation with the corresponding encoder feature map, preserving spatial detail that pooling alone would discard.

The output projection produces four channels, one for each class. Cropping is required by the unpadded architecture in the original paper; the submitted padded implementation concatenated matching feature-map sizes without that crop.

Low contrast between adjacent structures is a plausible source of segmentation error because local boundaries can become ambiguous. The coursework did not include a separate controlled low-contrast evaluation, so this remains a design consideration rather than a measured result.

Within the supplied validation examples, the selected configuration produced clear boundaries. The coursework did not establish performance across acquisition protocols or clinical readiness.

Findings

After examining the results provided by FCN with varying hyper-parameters and comparing these findings to the products of the U-Net implementation, we came to the conclusion that the better approach for this problem would be to apply a U-Net model. This is backed by the dice score results of each model: we noticed a great difference in performance between the two models, with U-Net scoring between 0.87 - 0.9, whereas FCN would score in the range 0.44 - 0.5. This makes U-Net the clear contender for this image segmentation problem.

Implementation of U-Net

The initial model was based on using the same U-Net architecture as described in the *U-Net: Convolutional Networks for Biomedical Image Segmentation* paper. This consists of 4 downsampling blocks followed by 4 upsampling blocks. Both the downsampling and upsampling blocks use a double convolution so we implemented a class for it to reduce repeated code.

The block contains two 3 x 3 convolutions with stride 1 and padding 1, each followed by batch normalisation and ReLU. Padding preserves width and height within the block so that encoder and decoder feature maps can be concatenated without the crop used in the original unpadded U-Net.

Batch normalisation has also been applied to normalise the outputs after each convolution to stabilize the learning process.

```
class DoubleConv(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
        )
    def forward(self, inputs):
        return self.block(inputs)
```

Each encoder block applies 2 x 2 max pooling followed by a double-convolution block. Each decoder block applies a transposed convolution first, concatenates the upsampled output with the corresponding encoder features, and then applies a double-convolution block.

```
class Down(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.MaxPool2d(kernel_size=2, stride=2),
            DoubleConv(in_channels, out_channels),
        )

    def forward(self, inputs):
        return self.block(inputs)

class Up(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.up = nn.ConvTranspose2d(
            in_channels, in_channels // 2, kernel_size=2, stride=2
        )
        self.block = DoubleConv(in_channels, out_channels)

    def forward(self, inputs, skip):
        inputs = self.up(inputs)
        return self.block(torch.cat((skip, inputs), dim=1))

class OutConv(nn.Module):
    def __init__(self, in_channels, num_classes):
        super().__init__()
        self.projection = nn.Conv2d(in_channels, num_classes, kernel_size=1)

    def forward(self, inputs):
        return self.projection(inputs)
```

The complete model combines an initial double-convolution block, four encoder blocks in the contracting path, four decoder blocks in the expanding path, and a final 1 x 1 convolution that projects the features to four class logits.

```
class UNet(nn.Module):
    def __init__(self, in_channels=1, num_classes=4):
        super().__init__()
        self.inc = DoubleConv(in_channels, 64)
        self.down1 = Down(64, 128)
        self.down2 = Down(128, 256)
        self.down3 = Down(256, 512)
        self.down4 = Down(512, 1024)
        self.up1 = Up(1024, 512)
        self.up2 = Up(512, 256)
        self.up3 = Up(256, 128)
        self.up4 = Up(128, 64)
        self.outc = OutConv(64, num_classes)

    def forward(self, inputs):
        x1 = self.inc(inputs)
        x2 = self.down1(x1)
        x3 = self.down2(x2)
        x4 = self.down3(x3)
        x5 = self.down4(x4)
        x = self.up1(x5, x4)
        x = self.up2(x, x3)
        x = self.up3(x, x2)
        x = self.up4(x, x1)
        return self.outc(x)
```

Experiments

Once we gained our implemented model that delivered satisfactory results, we went on to tune a group of its hyper-parameters to assess whether there could be any further improvements made to the segmentation results. From all the hyper-parameters, we found two groups that had the most significant effect on the model: one group impacted the structure through parameters such as number of layers, activation functions, and dropout layers and the other refined the loss and optimiser functions used with specific values in their parameters.

In an attempt to enhance the structure of the model, we first trained it with varying numbers of layers ranging from 2 to 5. At each layer, two activation functions are implemented: ReLU and sigmoid. The most successful model was then tweaked by adding a dropout layer after each double convolution with different values to reduce overfitting.

Depth	Activation	Regularisation	LR	Train loss	Val loss	Dice
4	ReLU	dropout = 0.1	0.01	0.07	0.07	0.89
4	ReLU	dropout = 0.1	0.005	0.02	0.08	0.89
4	ReLU	none	0.01	0.02	0.7	0.89
4	ReLU	dropout = 0.2	0.01	0.02	0.07	0.88
2	ReLU	none	0.01	0.019	0.13	0.86
3	ReLU	none	0.01	0.02	0.08	0.86
4	tanh	none	0.01	0.03	0.099	0.86
5	ReLU	none	0.01	0.03	0.08	0.86
3	sigmoid	none	0.01	0.03	0.12	0.82
4	sigmoid	none	0.01	0.04	0.11	0.81
2	sigmoid	none	0.01	0.02	0.17	0.78
4	ReLU	dropout = 0.3	0.01	0.05	0.23	0.75
4	sigmoid	dropout = 0.2	0.01	0.06	0.18	0.73

Table 1: Model structure hyper-parameter tuning

The four-level model performed best in the reported structure experiments, although depth had a limited effect on Dice. ReLU outperformed sigmoid in the recorded comparisons. Dropout values of 0.1 and 0.2 did not improve on the base model, while 0.3 underfit the supplied data.

We next compared cross-entropy, MSE, L1, and NLL loss with Adam, Adagrad, ASGD, and SGD, along with selected optimiser settings. Table 2 records substantially stronger Dice for cross-entropy with Adam than for the other tested loss-optimiser combinations. Adam at learning rate 0.001 produced 0.880 in the base comparison; the recorded run with `amsgrad=True` and `weight_decay=1e-8` reached approximately 0.889. That configuration is also the one used by the final surviving training notebook. The results are historical observations from the supplied validation split rather than a controlled estimate of variability across repeated runs.

Table 2: Loss and optimiser hyper-parameter tuning

Criterion	Optimiser	Parameters	Dice
Cross-Entropy	Adam	lr=0.001, amsgrad=True, weight_decay=1e-8	0.889
Cross-Entropy	Adam	lr=0.001, betas=(0.9, 0.999)	0.880
Cross-Entropy	Adam	lr=0.001	0.880
Cross-Entropy	Adagrad	lr=0.001	0.859
Cross-Entropy	Adam	lr=0.005	0.853
Cross-Entropy	ASGD	lr=0.005, weight_decay=0	0.779
Cross-Entropy	Adagrad	lr=0.005, lr_decay=0.1	0.777
Cross-Entropy	Adam	lr=0.001, amsgrad=True, weight_decay=1e-2	0.776
Cross-Entropy	Adagrad	lr=0.001, lr_decay=0.1	0.774
NLLLoss	Adagrad	lr=0.01, lr_decay=0.1, weight_decay=1e-8	0.472
NLLLoss	Adagrad	lr=0.01, lr_decay=0.05, weight_decay=0.05	0.455
NLLLoss	Adam	lr=0.001, amsgrad=True, weight_decay=1e-8	0.429
NLLLoss	Adagrad	lr=0.001, lr_decay=0.05, weight_decay=0.05	0.428
NLLLoss	Adam	lr=0.001, amsgrad=True	0.427
NLLLoss	Adam	lr=0.004, amsgrad=False, weight_decay=1e-6	0.427
NLLLoss	Adagrad	lr=0.001, lr_decay=0.1	0.421
NLLLoss	Adam	lr=0.004, amsgrad=True, weight_decay=1e-6	0.417
Cross-Entropy	ASGD	lr=0.005	0.407
Cross-Entropy	SGD	lr=0.001	0.358
L1	SGD	lr=0.01, momentum=0.9	0.171
L1	Adam	lr=0.001, amsgrad=True	0.159
MSE	ASGD	lr=0.005, weight_decay=0	0.122
MSE	Adagrad	lr=0.001, lr_decay=0.1	0.122
MSE	SGD	lr=0.001, momentum=0.9	0.121
MSE	SGD	lr=0.001	0.121
MSE	Adagrad	lr=0.001	0.121
L1	ASGD	lr=0.001	0.114
MSE	Adam	lr=0.001, amsgrad=True	0.114
MSE	Adam	lr=0.0001, amsgrad=True	0.112
MSE	Adam	lr=0.01, amsgrad=True	0.109
MSE	Adam	lr=0.001, amsgrad=True, weight_decay=1e-6	0.109
MSE	Adam	lr=0.001, amsgrad=True, weight_decay=1e-8	0.108
Optimiser	Default parameter values		
Adam	lr=0.001, betas=(0.9, 0.999), eps=1e-8, weight_decay=0, amsgrad=False		
Adagrad	lr=0.01, lr_decay=0, weight_decay=0, initial_accumulator_value=0, eps=1e-10		
ASGD	lr=0.01, lambd=0.0001, alpha=0.75, t0=1000000.0, weight_decay=0		
SGD	lr required; momentum=0, dampening=0, weight_decay=0, nesterov=False		

Conclusion

The final surviving notebook used the base four-level U-Net with cross-entropy loss and Adam at a learning rate of 0.001, with `amsgrad=True` and `weight_decay=1e-8`, for 100 epochs.

We varied depth, activation, and dropout, but none of those tested changes improved on the selected four-level ReLU U-Net under the coursework validation setup.

We also compared loss functions, optimisers, and optimiser settings. The strongest recorded configuration used cross-entropy and Adam at a learning rate of 0.001 with `amsgrad=True` and `weight_decay=1e-8`. The final notebook recorded a peak validation foreground Dice of 0.8882506773 at epoch 72.

Across the recorded experiments, validation Dice plateaued near 0.90 and longer training began to overfit the supplied validation split. This is an observation about the coursework data and protocol, not a general performance bound.

Overall, U-Net was the stronger architecture under the supplied coursework split. The experiment did not establish clinical readiness or performance across acquisition protocols or patient populations.

References

Huan, T. (2020, May 17). *simple-pytorch-fcn*. GitHub.

<https://github.com/tangh/simple-pytorch-fcn>

Lambda, H. (2019, February 17). *Understanding Semantic Segmentation with UNET* / by Harshall Lamba. Towards Data Science. Retrieved November 28, 2021, from <https://towardsdatascience.com/understanding-semantic-segmentation-with-unet-6be4f42d4b47>

PyTorch. (n.d.). *torch.optim — PyTorch 1.10.0 documentation*. PyTorch. Retrieved November 28, 2021, from <https://pytorch.org/docs/stable/optim.html>

Ronneberger, O., Fischer, P., & Brox, T. (2011). *U-Net: Convolutional Networks for Biomedical Image Segmentation*. Computer Vision Group, Freiburg. Retrieved November 19, 2021, from <https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/>

Tsang, S.-H. (2018, October 5). *Review: FCN — Fully Convolutional Network (Semantic Segmentation)* / by Sik-Ho Tsang. Towards Data Science. Retrieved November 23, 2021, from <https://towardsdatascience.com/review-fcn-semantic-segmentation-eb8c9b50d2d1>